Ders 4 Genişletilmiş Notu: Prototype, Singleton, Adapter, Bridge ve Composite Kalıpları

Ders 4 Genişletilmiş Notu: Prototype, Singleton, Adapter, Bridge ve Composite Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Prototype Kalıbı: Amaç ve Nota Programı Örneği
- 2. Prototype'ın Kalıp Yapısı ve Uygulanabileceği Anlar
- 3. Shallow Copy ve Deep Copy
- 4. Örnek Gerçekleme: Oyun Level Editörü
- 5. Singleton Kalıbı
- 6. Adapter Kalıbı
- 7. Bridge Kalıbı: Amaç ve Pencere Örneği
- 8. Bridge'in Kalıp Yapısı ve Uygulanabileceği Anlar
- 9. Örnek Gerçekleme: İki Çizim Kütüphanesiyle Şekiller
- 10. Composite Kalıbı: Amaç ve Grafik Gruplama Örneği
- 11. Composite'in Kalıp Yapısı, Gerçekleme Ayrıntıları ve Zayıf Yönleri
- 12. Örnek Gerçekleme: Makine Üretim Fabrikası
- Örnek Sorular
- Dersin Toparlaması

1. Prototype Kalıbı: Amaç ve Nota Programı Örneği

Ders, yaratımsal (creational) kalıplardan kalan Prototype ile başlar. Önceki derste Factory Method ve Builder görülmüştü.

1.1 Amaç

Prototype kalıbında yeni bir nesne oluşturmak için sınıfı sıfırdan örneklemek yerine, temel olarak kullanılacak bir prototip nesne tutulur. Yeni nesneler bu prototipin kopyası alınıp özellikleri değiştirilerek oluşturulur.

Hoca bunun ne işe yaradığını şöyle açıkladı: nesneler çok "ağır sıklet" ise, yani çok fazla durum bilgisi varsa ve bunların hepsini ilklendirmek uzun sürüyorsa (kurucuya aşırı sayıda parametre geçmek gerekiyorsa), bütün üye alanların listesini vererek sıfırdan nesne oluşturmak yerine var olan bir nesnenin kopyasını alıp yalnız bir iki alanının değerini değiştirmek daha kolaydır.

1.2 GoF örneği: nota yazma programı

Dörtlü çetenin (GoF) örneğinde müzik hobimiz olduğunu ve nota yazmaya yönelik bir program hazırlamak istediğimizi düşünüyoruz. Programı sıfırdan yazmayacağız; elimizde hazır bir grafik kütüphanesi (çerçeve program) var. Slayttaki şema (ekrandan, 00:10:00) şöyledir:

```
Tool (soyut)                        Graphic (soyut)
  Manipulate()                        Draw(Position)
```

```
   |-- RotateTool                        Clone(): Graphic
   |     Manipulate()                     |-- Staff
   |-- GraphicTool ----prototype---->     |     Draw(Position), Clone()
         Manipulate()                     |-- MusicalNote (soyut)
                                                |-- WholeNote: Draw(Position), Clone()
                                                |-- HalfNote:  Draw(Position), Clone()
```

```
GraphicTool.Manipulate():
    p = prototype->Clone()
    while (user drags mouse) {
        p->Draw(new position)
    }
    insert p into drawing
```

```
WholeNote.Clone() / HalfNote.Clone(): return copy of self
```

Soyut Tool sınıfı, araç çubuğundaki düğmeleri modeller; düğmeye tıklayınca çizime bir eleman bırakılabilir. Bazı araçlar hazır gelir: döndürme (RotateTool), yeni grafik ekleme (GraphicTool); boyutlandırma ve taşıma gibi başka araçlar da olabilir, çünkü bir programda her düğme her işi yapmaz. Graphic, herhangi bir grafik nesnesini modeller; temel işleri nesneyi bir konuma çizmek (Draw) ve nesnenin kopyasını almaktır (Clone). Hoca, slaytta Clone()'un dönüş türünün yazılmadığını belirtip şemaya `Clone(): Graphic` olarak ekledi.

Staff, notaların üzerine yazıldığı beş çizgili portedir; o da sayfa düzenine göre belirli koordinatlara çizilecek bir grafik nesnesidir. Notalar soyut MusicalNote altında somut sınıflardır: tam nota, yarım nota gibi. Hoca, içi boş ve içi dolu, tek ve çift olmak üzere temel dört varyasyonu, ayrıca sol anahtarını ve notanın porte üzerindeki konumuna göre farklı bir sese karşılık geldiğini hatırlattı.

Asıl problem şudur: çerçeve program nota sınıfları hakkında önceden bilgi sahibi olamaz. Notalar sınırlı sayıda olsa da GraphicTool, bilmediği sınıflardan nesneyi nasıl oluşturacak? Üstelik çerçeve yalnız müzik için değil, daha da genişletilebilir olacak şekilde, yani daha önce kimsenin düşünmediği alanlara uzanabilecek şekilde tasarlanıyor.

1.3 Kalıbın çözümü

Çözüm, şemadaki yeşil çerçeveyle işaretlenen kısımdır (ekrandan, 00:10:00): çerçeve yalnız Graphic'in Draw ve Clone metotlarına, araçlar da Manipulate metoduna dayanır. Üzerinde uğraşılacak her nesne Draw ve Clone'u desteklediği sürece grafik çerçevesi istenen alana genişletilebilir.

GraphicTool'un Manipulate metodu önce ilişkide olduğu prototip nesnesinin bir kopyasını oluşturur. Kullanıcı fareyi sürükledikçe yeni nesne o konumda çizilir; fare bırakılınca nesne çizime eklenir. Hoca bunu sunum programlarındaki davranışa benzetti: düğmeye tıklayınca bir şekil oluşur, belki önce silik çizilir, sürükledikçe taşınır, bırakınca yerine yerleşir.

1.4 Diğer seçenekler

Hoca, başka çözümlerin de akla gelebileceğini söyledi:

- **Factory Method:** her nota farklı olduğu için her biri için ayrı bir factory metodu gerekir. Tek bir fabrika sınıfıyla çözülebilir, ama nota türü kadar metot olması gerekir.
- **Abstract Factory:** aileler ve üyeler nasıl çizilecek? Örneğin tam notanın yedi versiyonu (do, re, mi, fa, sol, la, si), yarım notanın yedi versiyonu gibi. Bu da sonunda aşırı sayıda sınıf demektir.

Bu yüzden çok fazla fabrika metodu ya da sınıfı ortaya çıkacağı için bu seçenekler tercih edilmez. Hoca daha iyi bir çözüm bulan olursa paylaşmasını istedi.

2. Prototype'ın Kalıp Yapısı ve Uygulanabileceği Anlar

2.1 Kalıp yapısı ve bileşenleri

Slayttaki yapı (ekrandan, 00:14:00):

```
Client --------prototype--------> Prototype (soyut)
  Operation()                        Clone()
    p = prototype->Clone()            |-- ConcretePrototype1: Clone()  -> return copy of self
                                      |-- ConcretePrototype2: Clone()  -> return copy of self
```

- **Prototype:** nesnelerin kendisini klonlaması için bir arayüz sunar. Soyut bir sınıftır ve soyut bir Clone metodu vardır. Prototiplerin yapabileceği public işler için ek metotlar da olabilir.
- **ConcretePrototype:** kullanılacak nesnelerdir. İşlerinin metotlarına ek olarak kendisinin bir kopyasını döndüren bir Clone metodu bulunur.
- **Client:** bir prototipin kendisini kopyalamasını isteyerek yeni bir nesne oluşturur. Kendisine bir prototip verildiğinde onu istediği kadar klonlar ve klonlara istediği işleri yaptırır.

Avantajı şudur: yalnız üstteki soyut sınıf üzerinden çalışıldığında alt somut ürünlerin tam olarak nasıl oluşturulup birleştirildiğine müdahale etmek gerekmez; bu iş o sınıfların içinde kalır.

2.2 Uygulanabileceği anlar

Hocaya göre aşağıdaki durumlardan en az biri geçerliyse, "bunun yerine Factory Method kullansaydık daha iyi olmaz mıydı?" sorusunun yanıtı "hayır" olur ve Prototype kullanmak daha isabetlidir:

- Hangi sınıftan nesne oluşturulması gerektiği çalışma anında belirlenecekse.
- Ürünlerin sınıf hiyerarşisine paralel bir fabrika metodu ya da fabrika sınıfı hiyerarşisi kurulmak istenmiyorsa.
- Ürün sınıfının örneklerinin durum uzayı sınırlıysa, yani az sayıda farklı nesne olacaksa.

2.3 Reflection ve late binding

Hoca, Prototype'ın çok yaygın bir kalıp olmadığını, daha çok sınıfların doğrudan kodlanıp ele alınabileceği yapıların olmadığı dillerde işe yaradığını söyledi. Normalde sınıf kodlanır, sonra nesneler oluşturulup kullanılır. Çalışma anında daha "egzantrik" işler yapmak, örneğin bir nesnenin sınıf kodunu bilmeden ait olduğu sınıfın metotlarını, parametrelerini, bağımlılıklarını sorgulamak, hatta çalışma anında yeni özellikler eklemek istenebilir. Buna **reflection** denir. Kullanılan dilin böyle bir özelliği yoksa ve bu özellikler işe yarayacaksa, Prototype kalıbıyla biraz daha uğraşarak benzer bir yetenek kodlanabilir.

İkinci durum **late binding**'dir: bir nesnenin türünü baştan belirlemeyip daha sonra ilk yapılacak atamayla belirlemek. Güçlü tipli diller buna genelde izin vermez. Hocanın örneği: vatandaşlardan kimlik numarası, diğer ülke vatandaşlarından pasaport numarası alınacak ve bunların yapıları farklı. Tek bir alan tanımlayıp iki ya da daha fazla farklı türden veriden birini aktaracak kod yazılıyorsa, oluşturulacak kişinin hangisi olduğu ancak verilen veriye göre çalışma anında belli olur. Dil bunu desteklemiyorsa çözüm Prototype üzerinden üretilebilir.

3. Shallow Copy ve Deep Copy

Klon oluştururken izlenebilecek iki yöntem vardır. Kopyalanan nesnenin üye alanları primitive ise sorun yoktur. Ama primitive olmayan üyeler varsa, onların da primitive olmayan üyeleri, onların da başka üyeleri olabilir.

- **Shallow copy:** yalnız ilk düzeyde kopyalayıp kesmek.
- **Deep copy:** bütün üyelerin üyelerinin üyelerini, nereye kadar gidiyorsa kopyalamak.

Deep copy, özellikle bir çevrimsellik varsa sorun çıkarır. Hoca bunu Violet UML'de çizdiği bir Takım–Oyuncu şemasıyla gösterdi (ekrandan, 00:20:00): takım oyuncularını bilir (1'e çok ilişki), oyuncu da takımını bilir. Takımı deep copy ile kopyalarken bütün oyuncular kopyalanır, ama oyuncu da takımın farkındadır; takımı tekrar kopyalamamak için nesne kimliği, referans

ya da tekil tanımlayıcı gibi bir denetim gerekir. Şema daha da karmaşık olabilir: örneğin oyuncunun hangi takımın altyapısından geldiği de saklanıyorsa ve altyapı da takımın bir alt sınıfıysa deep copy daha da zorlaşır.

Hoca, hazır bazı yapıların bu işi kolaylaştırabileceğini, o durumda deep copy yapmanın daha mantıklı olabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Java'da `Object.clone()` varsayılan olarak shallow copy yapar ve sınıfın `Cloneable` arayüzünü işaretlemesini ister; işaretlenmemişse `CloneNotSupportedException` fırlatır. Bu yüzden derste görülen örnekte olduğu gibi clone metodunu kurucu çağırarak kendimiz yazmak, hangi alanın nasıl kopyalanacağını açıkça denetlemenin yaygın bir yoludur.

4. Örnek Gerçekleme: Oyun Level Editörü

4.1 Problem

Slayta göre (ekrandan, 00:25:30) bir bilgisayar oyunu için level editörü hazırlanıyor. Haritaya çok çeşitli türlerden canavarlar kolayca eklenmek isteniyor. Ama bir canavar oluşturmak kolay iş değil; bir sürü durum bilgisi var: HP, DEX, AGL, LUCK, CHR vb. Hoca, özellikle rol yapma oyunlarında bu alanların çok olabileceğini; örnekte beş alanla yetinildiğini, ama sağlık, kaçmadaki çeviklik, şans, karizma gibi daha fazlasının da konabileceğini söyledi. Örneğin kaynağı Head First Design Patterns kitabıdır.

Bütün alanları sıfırdan girerek canavar oluşturmak istenmiyor. Alanlara varsayılan değer atanabilir, ama dilin varsayılan değerleri (sıfır) işe yaramaz; oyunun zorluğuna göre çeşitli varsayılan değer kümeleri olabilir ve bölüm sonu olmayan diğer canavarlar, düşmanlar düşünüldüğünde çok sayıda varyasyon çıkar.

4.2 Sınıf şeması

```
«Abstract» Monster                  (Prototype rolü)
  # HP, DEX, AGL, LUCK, CHR : int
  # type : String
  + clone() : Monster {abstract}
  + getType() : String
  # Monster(int, int, int, int, int)
      ^
      | (kalıtım)
GenericMonster                      (ConcretePrototype rolü)
  + clone() : Monster
  + GenericMonster(int, int, int, int, int)

MonsterRegistry  --1-------*-->  Monster
  + findMonster( type : String ) : Monster
  + registerMonster( Monster )

MainApp (Client rolü) ---> MonsterRegistry, GenericMonster (bağımlılık)
```

(ekrandan, 00:25:30)

Monster soyut sınıfında type alanı canavarın türünü tutar: "kolay bölüm canavarı", "ork", "orta düzey bölüm canavarı ejderha" gibi adlar verilebilir. clone() soyuttur ve yeni bir Monster döndürür. Hoca bunun nedenini şöyle açıkladı: kimi durumda shallow copy, kimi durumda deep copy yapılacaktır ve buna ilgili somut alt sınıf karar verebilmelidir; her canavar tipi klonlama işini farklı tercih edebilir. Bütün alt sınıflarda ortak bir davranış tanımlanmak istenirse clone'a varsayılan bir gövde verilebilir, ama kardeş alt sınıfların davranışları birbirinden çok farklı olabildiği için bu her zaman tercih edilmez.

Kurucu bilerek **protected** tanımlanmıştır. Herhangi birinin sıfırdan, kendi istediği gibi canavar oluşturması istenmez; yeni nesneler yalnız clone üzerinden oluşturulsun istenir. Hoca, kurucunun illaki protected olması gerekmediğini, ama alt sınıfların erişebilmesi için üst sınıftakinin en az protected olması gerektiğini; private yapılırsa (iç sınıf gibi kullanımlar dışında)

hiç erişilemeyeceğini söyledi. Bu, nesne oluşturmayı denetim altında tutmak için tercih edilebilecek bir yöntemdir. Her alan için getter ve setter metotları vardır.

GenericMonster, oluşturulacak yeni genel canavarları simgeleyen somut sınıftır. Kurucusu üst sınıfın kurucusunu çağırır ve tür adına "Generic Monster" verir; clone() kendi üye alanlarının değerlerini aynen vererek yeni bir örnek oluşturur.

4.3 Kayıt (registry) gereksinimi

Hoca, klonlanarak oluşturulan nesnelerin yönetiminin bir yerde olması gerektiğini vurguladı. Dörtlü çete bu durumu ayrıca tartışmaya gerek görmemiş, ama hocaya göre illaki gerekecektir. Bu yüzden modelde bir kayıt sınıfı vardır: tür adı verilerek ilgili canavarı bulup döndüren findMonster ve klonlanarak oluşturulan yeni canavarı kaydeden registerMonster metotları.

4.4 Kod

Slayttaki kod (ekrandan, 00:28:50, 00:30:00):

```
package dp.prototype.example;
public class GenericMonster extends Monster {
    public GenericMonster(int hp, int dex, int agl, int luck, int chr) {
        super(hp, dex, agl, luck, chr);
        type = "Generic Monster";
    }
    public Monster clone( ) {
        return new GenericMonster(HP, DEX, AGL, LUCK, CHR);
    }
}
public class MonsterRegistry {
    private HashMap<String, Monster> monsters;
    public MonsterRegistry() { monsters = new HashMap<String,Monster>(); }

    public Monster findMonster( String type ) {
        return monsters.get( type );
    }
    public void registerMonster( Monster m ) {
        monsters.put( m.getType(), m );
    }
}
public class MainApp {
    public static void main(String[] args) {
        MonsterRegistry reg = new MonsterRegistry();
        reg.registerMonster( new GenericMonster(10, 10, 10, 10, 10) );
        Monster mySpecialMonster =
                reg.findMonster("Generic Monster").clone();
        mySpecialMonster.setHP(100);
        mySpecialMonster.setType("Bölüm sonu canavarı");
        reg.registerMonster(mySpecialMonster);
    }
}
```

MonsterRegistry basitçe bir HashMap'tir: anahtar canavarın türü (String), değer Monster nesnesinin kendisidir. Kurucu veri yapısını ilklendirir. findMonster, HashMap'in get metoduyla anahtara karşılık gelen canavarı varsa döndürür, yoksa null döndürür. registerMonster, canavarın türünü anahtar, nesnenin kendisini değer olarak kaydeder.

MainApp'te önce kayıt oluşturulur ve istenen bütün varsayılan değerlerle genel bir yaratık kaydedilir. Bölüm sonu canavarı oluşturmak için kayıttan "Generic Monster" alınıp klonlanır, sağlık puanı 100'e çıkarılır, türü "Bölüm sonu canavarı" yapılır ve yeni tür olarak kaydedilir.

> **Ders sonrası not:** GenericMonster.clone() type alanını kopyalamaz; kurucu her zaman "Generic Monster" atar. Bu yüzden kayıttaki "Bölüm sonu canavarı" ileride

klonlanırsa klonun türü yeniden “Generic Monster” olur. Hoca bu ayrıntıya değinmedi; tür bilgisinin de korunması isteniyorsa clone içinde type da kopyalanmalıdır.

4.5 Ne kazanıldı?

Bu bölüm sonu canavarını 100, 10, 10, 10 gibi bir dolu parametre vererek oluşturmaktan kurtulduk; karşılığında kaydı ilklendirdik. Hoca, tek bir canavar için bunun işe yaramayacağını, ama oyunlarda yüzlerce, binlerce, on binlerce canavar olacağını vurguladı. On binlerce ayrı nesneyi en az onlar mertebesinde üye alanını vererek ilklendirmek başka, bir kopyasını alıp birkaç alanını değiştirmek başkadır. Hocaya göre Prototype çok karmaşık bir kalıp değildir.

5. Singleton Kalıbı

5.1 Amaç ve örnekler

Amaç: bir sınıftan yalnız bir tane nesne oluşturulsun, başka nesne oluşturulamasın ve her zaman bu nesneye erişilsin. Böyle bir ihtiyaçta Singleton kullanılır. Hocanın örnekleri:

- Bir bilgisayara istenildiği kadar yazıcı tanıtılabilir, ama yazdırma biriktiricisi (print spooler) süreci bir tanedir.
- Bilgisayarda birden fazla uygulama çalışabilir, ama görev yöneticisinin tek bir örneği olur.
- İstenildiği kadar dosya gezgini açılabilir, ama dosya yönetimi süreci (file manager) ve pencere yöneticisi birer tanedir; dosya yöneticisi çökerse bilgisayarı yeniden başlatmak gerekir.

5.2 İlk akla gelen çözüm ve sorunu

Bunun için illaki tasarım kalıbı gerekmez: tek olacak nesne static bir nesne olarak tanımlanabilir. Ama bu static üye hangi sınıfta olacak? Tek olması istenen nesne farklı sınıflar tarafından kullanılıyorsa üyenin hangisine konacağı sorun olur; nesneyle ilgili bir değişiklik gerektiğinde hem onu barındıran sınıfta hem ilişkili sınıflarda düzenleme yapmak gerekir.

5.3 Önerilen çözüm

Slayttaki çözüm (ekrandan, 00:33:30): bir sınıfa, kendisinin tek bir örneğinin olmasını sağlama sorumluluğunu atamak.

- Sınıfın kendi türünden static bir üyesi olur.
- Sınıfın kurucusu private tanımlanarak kurucuya erişim engellenir.
- Sınıfın bu üye için bir factory metodu bulunur.
- Kalıba adını veren bu sınıfın adı Singleton olarak belirlenmiştir.

```
Singleton
  - uniqueInstance : Singleton {static}
  - state : SingletonData
  - Singleton( ... )
  + getInstance( ) : Singleton {static}
  + singletonOperation( )
  + getterMethods( )

getInstance():
  if( uniqueInstance == null ) {
    uniqueInstance = new Singleton( ... )
  }
  return uniqueInstance;
```

uniqueInstance, sınıfın kendisiyle aynı türden ve static bir üyedir. Başka üye alanlar da olabilir (şemada state). Kurucu private olduğu için kimse Singleton nesnesi oluşturamaz. Nesneye erişmenin tek yolu static getInstance metodudur. Nesne alındıktan sonra singletonOperation diye özetlenen çeşitli işlemler yaptırılabilir, üyelere erişmek için getter metotları eklenebilir.

getInstance kritik metottur: önce uniqueInstance'ın daha önce oluşturulup oluşturulmadığına bakılır, oluşturulmamışsa metodun içinde oluşturulur; aksi halde var olan nesne döndürülür. Böylece yalnız tek bir nesne olur ve kurucuya başka hiçbir yoldan erişilemediği için başka Singleton nesnesi oluşturulamaz. Nesneye ulaşmak için yine nesnenin kendi sınıfı kullanılır, başka bir sınıfa gitmek gerekmez. Hoca, dönemin başında da tartışıldığı gibi, bir kalıbın illaki ayrıntılı bir çözüm olması gerekmediğini, bazen basit fikirlerin günü kurtardığını söyledi.

> **Ek bilgi (derste anlatılmadı):** Slayttaki "null ise oluştur" biçimindeki tembel (lazy) ilklendirme, çok iş parçacıklı programlarda iki iş parçacığı aynı anda null denetimini geçerse iki nesne oluşturabilir. Java'da bu genellikle getInstance'ı `synchronized` yaparak, nesneyi sınıf yüklenirken ilklendirerek ya da tek elemanlı bir `enum` kullanarak önlenir.

5.4 Singleton ne zaman gerçekten gerekir?

Hocaya göre bu tasarımın artısı, Singleton sınıfının alt sınıfları oluşturularak hem tekil kalan hem başka yetenekleri olan nesnelerin kolayca elde edilebilmesidir. Kalıbın en çok işe yaradığı yer, tekil nesneyi farklı farklı istemcilerin kullanmasıdır. Tekil veriye yalnız tek bir sınıf ulaşacaksa basit static üye alan yolu kullanılmalıdır; bu daha doğru bir iştir. Kalıp yeterince açık olduğu için hocanın ayrıca bir örneği yoktur.

Hoca ayrıca ileride görülecek State ve Strategy kalıplarındaki nesnelerin de Singleton adayı olduğunu söyledi.

Böylece yaratımsal kalıplar bitmiş olur ve GoF'un ikinci sırada anlattığı yapısal (structural) kalıplara geçilir.

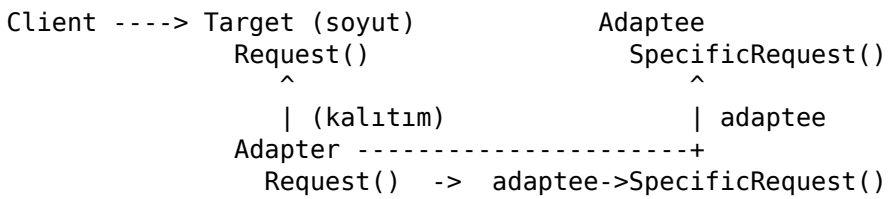
6. Adapter Kalıbı

6.1 Amaç ve priz örneği

Slayta göre (ekrandan, 00:40:00) amaç: işlev açısından uyumlu ancak metot adları açısından uyumsuz bir istekçi sınıf ile bir sunucu sınıfı birbirleriyle tek yönlü konuşturmak.

Kalıba adını veren durum elektrik prizleridir. Farklı ülkelerin farklı elektrik standartları vardır. Avrupa standardında belli aralıklı iki yuvarlak delik vardır. İngiliz standardında aynı aralıkta iki yuvaya ek olarak ikisinin üstünde ortada üçüncü bir yuva vardır; alttaki iki yuva bir kapakla kapalıdır, fişin daha uzun olan üçüncü bacağı önce girip içerideki dili aşağı itince kapaklar açılır ve diğer iki bacak girebilir. Bazı ülkelerde delikler birbirine daha yakın, bazılarında ince uzun dikdörtgendir. Başka bir ülkeye gidip telefonu şarj etmek isteyen biri seyahat adaptörü kullanır. Hoca, birkaç ülke standardına katlanır bacaklarla uyan bir adaptör görselini gösterdi: bir tarafına herhangi bir alet takılır, diğer tarafındaki bacaklardan bulunulan ülkeye uygun olan prize sokulur. Adapter kalıbı da aynı biçimde çalışır.

6.2 Kalıp yapısı



(ekrandan, 00:43:20)

Adaptee, asıl işi yapan, belli bir imzayla kodlanmış SpecificRequest metodu olan sınıftır. Client ise kendi kodu içinde Target sınıfından bir nesnenin Request metodunu çağırır. Ama o iş hâlihazırda SpecificRequest olarak kodlanmıştır. Çözüm: Target'ın bir alt sınıfı (Adapter) oluşturulur, asıl işi yapacak nesne ona üye alan olarak verilir ve Request metodunun içinde o nesnenin hazır metodu çağrılır.

6.3 Örnek: çizim programına hazır daire sınıfını eklemek

Slayttaki problem (ekrandan, 00:46:40): bir çizim programı üzerinde çalışılıyor ve programın bir kısmı hâlihazırda gerçeklenmiştir.

```
Shape (soyut)                          XXCircle
  +setLocation()                         +setLocation()
  +getLocation()                         +getLocation()
  +display()                             +displayIt()
  +fill()                 <--- ??? -->   +fillIt()
  +setColor()                            +setItsColor()
  +undisplay()                           +undisplayIt()
   |-- Point:  +display() +fill() +undisplay()
   |-- Line:   +display() +fill() +undisplay()
   |-- Square: +display() +fill() +undisplay()
```

Nokta, çizgi ve kare oluşturulmuştur; sıra daireye gelmiştir. Hoca, iki boyutlu bilgisayar grafiklerinde daire çizmenin kolay olmadığını anlattı: ekran satır ve sütunlardan oluşmuş karelerdir, eğimli bir çizgiyi ya da çemberi göze yumuşak gösterebilmek için bazı ara noktalara ara renk tonlarında noktacıklar konur. Bunu kodlamak masraflıdır. Daire kodunu sıfırdan yazmadan önce elde hazır bir XXCircle sınıfı olduğu hatırlanır; daireyi doldurmak için fillIt, rengini ayarlamak için setItsColor, ekrandan silmek için undisplayIt gibi gereken bütün işleri yapar. Ama mevcut tasarıma uymaz. Üstelik kaynak kodu kaybolmuş, yalnız derlenmiş hali eski yedeklerde bulunabilmiştir; bu yüzden kod değiştirilemez.

Çözüm Adapter'dır: yeni çerçeveye göre, Shape'ten türeyen kendi Circle sınıfımız oluşturulur ve içine bir XXCircle üyesi konur. Circle'ın her metodu XXCircle üyesinin ilgili metodunu çağırır; örneğin Circle.undisplay(), XXCircle'ın undisplayIt() metodunu çağırır. Bu örnekte Circle Adapter, XXCircle Adaptee rolündedir.

> **Ders sonrası not:** Hoca Shape'ten gelen ve uyulması gereken metot imzalarını "istemciniz gibi düşünebilirsiniz" diye anlattı. Kalıbın rol adlarıyla Shape, büyük olasılıkla Target rolündedir; Client ise Shape arayüzünü kullanan çizim programının kodudur.

7. Bridge Kalıbı: Amaç ve Pencere Örneği

7.1 Amaç

Arada uzun bir mola verildikten sonra (01:01:28) yapısal kalıplara Bridge ile devam edilir. Hocaya göre Bridge, amacının iddialı olmasından da anlaşılacağı gibi daha karmaşık kalıplardan biridir: bir nesnenin arayüzünü ve gerçeklemesini birbirinden ayırmak. Hoca, nesnenin veri ve metotlarıyla bir bütün olması ilkesinin burada bilerek esnetildiğini; kalıtımın değişik biçimlerde kullanılarak bazı noktalarda esneklik sağlandığını söyledi. Eski 32 bit Windows programlarını yazanların "handle" kavramıyla bunu aslında bildiğini ekledi.

7.2 GoF örneği: farklı masaüstü ortamlarında pencere

Elimizde bir GUI çerçeve programı var ve pencere soyutlamasından söz ediyoruz. Uygulamalar farklı masaüstü ortamlarını desteklesin isteniyor, ama ortamların pencere gereksinimleri farklıdır. İlk akla gelen: soyut bir Window sınıfı ve ondan kalıtımla XWindow, PMWindow, GNOMEWindow, KDEWindow gibi sınıflar.

Gereksinimler çeşitlendikçe bu çözümün esnek olmadığı ortaya çıkar. Örneğin pencereyi simge durumuna küçültmenin yanında, saatin olduğu sistem tepsisi alanına bir sürümünü koymak istenebilir; antivirüs programları ve donanım denetleyicileri genelde böyle çalışır, kapatıldıklarında bile arka planda bir bileşenleri çalışır ve tıklanınca ana pencere açılır. Bunun için yeni bir IconWindow sınıfı gerekir, ama her masaüstü ortamı için ayrı bir sürümü: XIconWindow, PMIconWindow, GNOMEIconWindow, KDEIconWindow. Dört ortam varsa simge pencereleri için ek dört sınıf kodlanır.

Hoca, bu problemin Abstract Factory ile de çözülebileceğini, ama orada ailelerin üyelerinin (düğme, radyo düğmesi gibi) işlev bakımından birbirinden tamamen ayrık olduğunu hatırlattı.

Burada ise hem pencere yeteneği hem de onun simge hali vardır ve ikisi arasında paylaşılan ortak kod olacaktır. Ortak işlemleri, yetenekleri, metotları nereye koyacağız? XWindow'daki bazı kodların XIconWindow'da da olmasını nasıl sağlayacağız? XIconWindow hem IconWindow'dan hem XWindow'dan kalıtım alabilir, ama XWindow'dan alacağı bazı metotlara simge penceresi olduğu için hiç ihtiyacı olmayabilir ve çoklu kalıtımı her dil desteklemez. Bu noktada Bridge devreye girer.

7.3 Bridge'in önerdiği çözüm

Temel fikir: pencere soyutlamaları ile pencerelerin gerçekten çizilmesini, yani gerçeklemelerini ayrı sınıf hiyerarşilerine koymak ve iki hiyerarşi arasında bir bağ kurmak (ekrandan, 01:10:00):

```
Window (soyut) ------imp------> WindowImp (soyut)
  DrawText()                      DevDrawText()
  DrawRect()                      DevDrawLine()
   |-- IconWindow                   |-- XWindowImp
   |     DrawBorder()               |     DevDrawText() -> XDrawString()
   |       -> DrawRect(), DrawText()|     DevDrawLine() -> XDrawLine()
   |-- TransientWindow              |-- PMWindowImp
         DrawCloseBox()                 DevDrawLine(), DevDrawText()
           -> DrawRect()

Window.DrawRect():
  imp->DevDrawLine()
  imp->DevDrawLine()
  imp->DevDrawLine()
  imp->DevDrawLine()
```

Window, soyut bir sınıftır ve bir WindowImp nesnesine sahiptir. Window'un somut alt sınıflarıyla pencerelerin ne yapması gerektiğinin hiyerarşisi kodlanır (simge penceresi, geçici pencere). Şeffaf pencere gibi bir gereksinim daha eklenseydi ilk çözümün ne kadar az esnek olduğu daha da göze batardı. Hoca, sıradan bir pencere için kalıbın daha çok bir ConcreteWindow alt sınıfı önerdiğini belirterek şemaya bunu ekledi.

Pencerelerin herhangi bir pencere yöneticisine göre nasıl gerçekleneceği ise WindowImp hiyerarşisinde tanımlanır. Bir pencereyi çizerken içine yazı yazmak gerekir (DrawText); sınırlarını çizmek için bir dörtgen çizen DrawRect vardır. DrawRect bütün işini imp üyesi üzerinden yaptırır: dört kez DevDrawLine çağırır. Çizginin o ortamda nasıl çizileceği somut alt sınıfta (örneğin XWindowImp'te XDrawLine ile) kodlanır.

IconWindow nesnesi, üst sınıfı sayesinde işlerini imp üyesi üzerinden yaptırdığı için, ona imp olarak bir XWindowImp verildiğinde X Window altında çalışan bir simge penceresi olur. Yazılım Presentation Manager ortamına taşınacağı zaman simge penceresine bir PMWindowImp nesnesi verilir ve artık o ortamda çalışır. İki kalıtım hiyerarşisi arasındaki ilişkiyi bu üye alan sağlar: soyutlama hiyerarşisinde, gerçekleme hiyerarşisi türünden bir nesne üye alan olarak tanımlanır.

> **Hocanın vurgusu:** GoF zamanında UML standardı oturmamıştı, önerilmemişti bile. UML standardında kalıtım okunun ucu üst sınıfa dayanır; şemalar çizilirken buna dikkat edilmelidir (ekrandan, 01:10:00).

8. Bridge'in Kalıp Yapısı ve Uygulanabileceği Anlar

8.1 Kalıp yapısı

```
Abstraction (soyut) -------imp-------> Implementor (soyut)
  Operation()                            OperationImp()
    -> imp->OperationImp();               |-- ConcreteImplementorA: OperationImp()
   |-- RefinedAbstraction                 |-- ConcreteImplementorB: OperationImp()
```

(ekrandan, 01:14:20)

Bir soyutlama hiyerarşisi ve bir gerçekleme hiyerarşisi vardır; her birinin somut alt sınıfları olur. Soyutlamada bütün operasyonlar gerçekleme üyesi üzerinden yaptırılır. Gerçeklemenin soyut sınıfında metot imzaları tanımlanır; bu haliyle bir interface de olabilir. Gerçeklemenin somut alt sınıflarında bu soyut işlemlerin gövdeleri doldurulur.

8.2 Uygulanabileceği anlar

- Bir soyutlama ile gerçeklemeleri arasındaki bağın sabit, kalıcı olması istenmediğinde; bağın dinamik olması istendiğinde.
- Soyutlamaların ve gerçeklemelerin ayrı ayrı özelleşmesi gerektiğinde. Bridge'in çözdüğü sorun ailesinin ortak özelliği budur.

Hoca, nesneye yönelimin temel ilkelerinden birini hatırlattı: **neyin değiştiğini bul ve onu sarmala.** Değişen şeyler genelde özelleşmiş sınıf kalıtım hiyerarşileriyle sarmalanır; o kadarına gerek yoksa o işe adanmış bir sınıf konarak sarmalanır.

Hoca ileride görülecek Strategy kalıbının da Bridge'e benzediğini, ama Strategy'de soyutlamanın alt sınıfları olmadığını, bu farkı orada ayrıca söyleyeceğini belirtti.

9. Örnek Gerçekleme: İki Çizim Kütüphanesiyle Şekiller

9.1 Problem

Hoca örneği bir kaynaktan aldığını, ama kaynağı şu an bulamadığını söyledi. Bir çizim programı üzerinde çalışılıyor. İlk aşamada dikdörtgenler desteklenecek; ama iki farklı dikdörtgen ailesi var ve farklı ailelerdeki dikdörtgenler farklı yöntemlerle çiziliyor. Bunun için iki hazır çizim kütüphanesi var: DP1 ve DP2.

Bir dikdörtgen çizmek dört çizgi çizmek demektir. Köşe noktalarının koordinatları (x_1, y_1), (x_1, y_2), (x_2, y_2), (x_2, y_1) biçimindedir; orijin noktası nereye konursa konsun fark etmez. Bu dört noktayı birleştiren dört çizgi çizilir. Ama iki kütüphanede çizgi çizme farklı kodlanmıştır:

- DP1: `draw_a_line(x1, y1, x2, y2)`.
- DP2: `drawline(x1, x2, y1, y2)`; koordinatların veriliş sırası farklıdır.

Elde, dikdörtgenin hangi türden olduğunu ve noktalarını bu sıralara göre veren bir metin ya da JSON dosyası vardır.

9.2 İlk çözüm

İlk akla gelen (ekrandan, 01:20:00): iki tip farklı dikdörtgen varsa bunlar bir üst sınıftan kalıtımla türetilir ve çizim metotları istenen iki farklı şekilde gerçeklenir.

```
Client ----> Rectangle (soyut)
               +draw()
               #drawLine()
                |-- V1Rectangle: #drawLine()  ---->  DP1: +draw_a_line()
                |-- V2Rectangle: #drawLine()  ---->  DP2: +drawline()
```

Slayttaki açıklamaya göre protected metodun amacı şudur: drawLine metodu nesnenin iç çalışmasıyla ilgilidir, o yüzden public yapılmamalıdır. Private metotlar kalıtımla aktarılır ama alt sınıfta doğrudan kullanılamaz; protected metotlar hem farklı türden nesnelere görünmez hem de alt sınıflara kalıtımla aktarılır. Hoca drawLine'ın çok alt düzeyde çizgi çizmeyi kodlayacağını anlattı: çizgiler illaki dikey ya da yatay değildir, göze yumuşak gelmesi için ara noktalara hafif tonlarda noktacıklar konabilir; ya da kütüphanelerden biri DirectX, öbürü OpenCL gibi bir yöntemle ekran kartını kullanıyor olabilir ve bu çağrılar birbirinden çok farklıdır. Bu iç ayrıntılar protected drawLine'da kalır; draw metodu yalnız köşe noktalarıyla drawLine'ı çağırır.

9.3 Yeni gereksinimler ve sınıf patlaması

Yeni bir gereksinim gelir: artık daireler de çizilecek ve onlar da iki aileden olacak. Birinci dikdörtgen ailesiyle birinci daire ailesi DP1'i, ikinci aileler DP2'yi kullanacak. Kütüphanelerde daire çizme metotları zaten vardır, ama programımızda şekil olarak yoktur. Program şeklin

dikdörtgen mi, üçgen mi, daire mi olduğunu dert etmeden bütün şekilleri eş biçimde yönetebilsin istenir. Bu yüzden ikinci çözümde soyut bir Shape üst sınıfı eklenir; Rectangle ve yeni Circle ondan türer, Circle'dan da V1Circle ve V2Circle türer. Hoca, eklenmesi gereken ilişkilerin epey arttığını gösterdi: iki aileden iki şekil, iki kere iki dört somut sınıf.

Şimdi bir üçgen eklenirse Triangle, V1Triangle ve V2Triangle olmak üzere üç sınıf daha gerekir. Üçüncü bir çizim kütüphanesi DP3 eklenirse V3Rectangle, V3Circle, V3Triangle sınıfları gerekir. Mevcut çözümü yeni bir şekil ya da yeni bir kütüphane ile zenginleştirmek istediğimizde çok sayıda sınıf eklemek ve kodda çok yerde değişiklik yapmak zorunda kalırız.

9.4 Bridge ile çözüm

Sorunun temel kaynağı şudur: farklı şekil türleri ve farklı çizim yöntemleri vardır, ama bunlar iç içe modellenmiştir. Şekil türleri ile çizim yöntemleri birbirinden ayrılabilirse ikisi ayrı ayrı özelleştirilebilir. Bridge'in önerdiği çözüm budur (ekrandan, 01:25:40):

```
Shape Abstraction                        Drawing Implementation
Shape (soyut) ------_dp------------->    Drawing (soyut)
  +draw()                                  +drawLine()
  #drawLine()                              +drawCircle()
  #drawCircle()                             |-- V1Drawing: +drawLine() +drawCircle() ---> DP1:
   |-- Rectangle: +draw()                   |-- V2Drawing: +drawLine() +drawCircle() ---> DP2:
   |-- Circle:    +draw()
```

Shape'ten Rectangle ve Circle somut alt sınıfları türer. Soyut Drawing sınıfında drawLine, drawCircle gibi metotlar vardır; V1Drawing ve V2Drawing somut alt sınıfları bu metotların gövdelerini, ilişkili oldukları kütüphanenin metotlarını çağırarak doldurur. Shape'ten Drawing'e bir _dp üye alanı konur.

Bu çözüme üçgen eklemek için Triangle basitçe Shape'in alt sınıfı olarak eklenir; herhangi bir kütüphaneyle ilişkilendirmeye gerek yoktur, çünkü o ilişki üst sınıftaki _dp üyesi üzerinden zaten vardır. Üçüncü bir kütüphane eklemek için V3Drawing sınıfı Drawing'in kardeşi olarak eklenir ve DP3'ü kullanır. Önceki tasarımda üçüncü aileyi desteklemek için her şekil için ayrı bir V3 sınıfı gerekiyordu; şimdi yeni gerçekleme tek başına tasarıma dahil edilir ve diğer sınıflarda hiçbir değişiklik gerekmez. Hocaya göre Bridge bu belirli probleme çok odaklanmış, güzel bir çözüm sağlar.

Hoca, orijinal şemada _dp üyesinin çizilmesinin unutulduğunu, ancak koddan görülebildiğini söyledi.

9.5 Kod

Slayttaki kod (ekrandan, 01:30:00, 01:32:20):

```
package bridge.solution2;
public abstract class Shape {
   public abstract void draw( );
   private Drawing _dp;
   public Shape (Drawing dp) { _dp= dp; }
   protected void drawLine(double x1, double y1,
     double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
   protected void drawCircle(double x,double y,
     double r ) {_dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
   private double _x, _y, _r;
   public Circle (Drawing dp,
               double x,double y,double r) {
      super( dp) ;
      _x= x; _y= y; _r= r ;
   }
   public void draw () { drawCircle(_x,_y,_r); }
}
```

```
public abstract class Drawing {
   public abstract void drawLine( double x1, double y1,
             double x2, double y2 );
   public abstract void drawCircle (double x,double y,
             double r);
}
public class V1Drawing extends Drawing {
   public void drawCircle(double x, double y, double r) {
      DP1.draw_a_circle(x,y,r);
   }
   public void drawLine(double x1, double y1,
             double x2, double y2) {
      DP1.draw_a_line(x1,y1,x2,y2);
   }
}
```

Shape soyuttur ve draw metodu soyuttur. Shape mutlaka bir Drawing üyesiyle ilişkilidir; kurucuya çizim gerçeklemesi mutlaka verilir, çünkü gerçeklemesiz bir soyutlama tek başına olmaz. drawLine çağrıldığında, Shape hangi Drawing ile ilişkiliyse çizgiyi o çizer; daire çizmek de aynıdır.

Circle'ın orijin noktası ve yarıçapı vardır. Kurucuda önce hangi çizim ailesiyle ilişkili olduğu verilir, sonra diğer üye alanlar atanır. draw() üst sınıfın drawCircle metodunu çağırır. Hoca bunun önemini vurguladı: alt sınıf nesneleri gerçeklemeyle doğrudan oynamak zorunda kalmaz (bunun yan etkileri olabilirdi), yalnız üst sınıftan yararlanarak işini yaptırır. Rectangle da benzer şekilde Shape'ten türer, x ve y koordinatlarını kurucuda çizim kütüphanesiyle birlikte alır ve çizim zamanı üst sınıftan aldığı metotlarla köşe noktalarını birleştiren dört çizgiyi çizdirir. V2Drawing de her işini DP2 kütüphanesine yaptırır. Hoca, derleme hatası olmasın diye DP1 ve DP2 kütüphanelerinin iskelet kodunu da projeye eklediğini söyledi.

> **Hocanın vurgusu:** Bridge'de soyutlama hiyerarşisinin üst sınıfı (Shape) interface olamaz, çünkü üye alan ve gövdesi dolu metotlar kodlamak gerekir. Hoca önce gerçekleme hiyerarşisinin de yalnız interface ile kurtarılamayacağını söyledi (01:30:25), sonra Drawing'in metotlarının hiçbirinin gövdesini doldurmak gerekmediğini görüp bu sözünü geri aldı (01:31:49): gerçekleme hiyerarşisinin soyut üst sınıfı bir interface olarak da kullanılabilir.

9.6 Değerlendirme

"Neyin değiştiğini bul ve onu sarmala" ilkesi burada akıllıca uygulanmıştır: aslında birden fazla şeyin değiştiği, bunların soyutlama ve gerçekleme olarak iki ayrı ana kategori olduğu görülmüş ve her biri ayrı kalıtım hiyerarşisine konmuştur. Şekil soyutlaması çizim gerçeklemesinden ayrılmış; biri diğerinden kalıtımla türetilmemiş, biri diğerine üye alan olarak verilmiştir (sahiplik ilişkisi). Şekiller ve çizim türleri aynı sınıflarda toplandığında bağlaşım çok artıyor ve her yeni şekil ya da çizim türü için çok sayıda sınıf ekleniyordu; Bridge ile bu yan etkiler ortadan kalkmıştır.

10. Composite Kalıbı: Amaç ve Grafik Gruplama Örneği

Hoca sırada Decorator olduğunu, ama ders geç başladığı için Composite'e geçtiğini söyledi.

10.1 Amaç

Composite, çeşitli bileşenlerden oluşan bileşik anlamına gelir. Kalıbın amacı, hiyerarşi şeklindeki parça-bütün ilişkilerini desteklemektir; böylece hiyerarşideki tek tek nesneler ve onların birlikte oluşturduğu varlık istemci tarafından tamamen aynı biçimde görülür.

Gerçek dünyadan örnekler: bir ordu kolordulara, kolordular tümenlere, tümenler tugaylara, tugaylar alaylara ayrılır. Bir üniversite önce rektörlük düzeyinde düşünülür, fakültelere, fakülteler bölümlere, bölümler anabilim dallarına ayrılır.

10.2 GoF örneği: grafik gruplama

GoF örneğinde yine bir grafik çizim programı vardır. Oluşturulan bir resim kendi içinde çizgiler, dörtgenler, metinler gibi grafik bileşenler içerir; bunlardan bazıları alt resimler olarak düşünülebilir. Örneğin bir resim bir alt resim, bir çizgi ve bir dikdörtgen içerebilir; alt resim de metin, çizgi ve dikdörtgen içerebilir.

Hoca bunu bir sunum programında gösterdi: bir dikdörtgen, bir daire ve bir üçgen çizdi; dikdörtgen ile üçgeni belli bir noktada hizalayıp seçti ve sağ tıklayıp grupladı. Artık birine tıklayınca ikisi birden seçilir, daire ise ayrı durur. Ana resim, daireyi ve (üçgen ile dikdörtgenden oluşan) alt resmi içerir; böylece hiyerarşik bir yapı kurulmuş olur.

10.3 Çözüm

Resim, metin, dikdörtgen gibi nesneler bir araya getirilip bir koleksiyon oluşturulabilsin ve farklı çizim bileşenleri ortak biçimde ele alınabilsin diye soyut bir Graphic sınıfı tanımlanır ve bütün sınıflar ondan türetilir (ekrandan, 01:40:00):

```
Graphic (soyut) <-------------graphics------------+
  Draw()                                          |
  Add(Graphic)                                    |
  Remove(Graphic)                                 |
  GetChild(int)                                   |
   |-- Line:      Draw()                          |
   |-- Rectangle: Draw()                          |
   |-- Text:      Draw()                          |
   |-- Picture  <>--------------------------------+
         Draw()           -> forall g in graphics: g.Draw()
         Add(Graphic g)   -> add g to list of graphics
         Remove(Graphic)
         GetChild(int)
```

Graphic'te çizim için Draw, alt parça eklemek için Add, belirli bir alt parçayı silmek için Remove, belirli numaralı bir çocuğa (birinci, ikinci, üç yüzüncü bileşen gibi) erişmek için GetChild metotları vardır. Hepsi soyuttur, çünkü gövdeleri çizim elemanının türüne göre çok farklı olacaktır.

Picture kendi içinde grafik bileşenleri tutar; kendini çizer, bileşenlerine yenisini ekler, siler, belirli bir çocuğu getirir ve soyut metotların hepsinin gövdesini kodlamak zorundadır. Picture'ın Draw metodu listedeki her elemanın tek tek çizilmesini sağlar; elemanlardan biri alt resimse o da kendi alt bileşenlerinin çizilmesini sağlar. Kaç düzey hiyerarşi kurulursa kurulsun çizim isteği en alta kadar iletilir ve bütün şekiller ekrana çizilir.

Line, Rectangle ve Text sınıflarında ise yalnız Draw anlamlıdır. Hoca bunu kalıbın en önemli eleştirisi olarak vurguladı: resim ve alt bileşenleri eş biçimde ele alabilmek için soyut üst sınıf ve oradaki bütün ayrıntılı metotlar gereklidir; ama bir çizginin alt bileşeni olmayacağı için ona ekleme ya da silme yapmak veya "şu numaralı çocuğu getir" demek anlamsızdır. Bu metotların gövdeleri boş bırakılır; getter gibi olanlar null ya da 0 gibi bir şey döndürmek zorundadır, başka yolu yoktur.

11. Composite'in Kalıp Yapısı, Gerçekleme Ayrıntıları ve Zayıf Yönleri

11.1 Kalıp yapısı ve katılımcılar

Slayttaki yapı (ekrandan, 01:50:00):

```
Client ----> Component (soyut) <-----------children----------+
               Operation()                                   |
               Add(Component)                                |
               Remove(Component)                             |
               GetChild(int)                                 |
                |-- Leaf: Operation()                        |
                |-- Composite <>-----------------------------+
```

```
                    Operation()  -> forall g in children: g.Operation();
                    Add(Component), Remove(Component), GetChild(int)

Nesne yapısı örneği:
aComposite
  |-- aLeaf
  |-- aLeaf
  |-- aComposite
  |     |-- aLeaf
  |     |-- aLeaf
  |     |-- aLeaf
  |-- aLeaf
```

- **Component:** nesneler topluluğunun katılımcıları için ortak kullanım arayüzü sunar; bir bütünün parçaları için erişim ve düzenleme metotları tanımlar. Yapılması gereken operasyonlara ek olarak kalıbın dayattığı kritik metotlar: yeni alt bileşen eklemek için Add, belirli bir alt bileşeni silmek için Remove ve isteğe bağlı olarak belirli sıradaki çocuğa erişmek için GetChild.
- **Leaf:** topluluktaki parçaları simgeler; hiyerarşik yapının en altındaki düğümlerdir, çocukları olamaz. Adı ağaç terminolojisinden gelir: alt düğümü olmayan düğümlere yaprak denir.
- **Composite:** hiyerarşik yapının ara düğümleridir. Component'ten türer, somut bir sınıftır; Component türünden bir çocuk listesi tutar. Bir işlem yapılacağı zaman o işlemin bütün alt bileşenlere iletilmesi için bir "for each" döngüsünde işlemi çocuklara gönderir. Farklı tür ara düğümler varsa bunlar Composite'in alt sınıfları olarak modellenir.
- **Client:** parçaları ve toplulukları benzer biçimde kullanabilir. Bütün operasyonlar Component'te tanımlı olduğu için ister kök üzerinden, ister bütünün herhangi bir parçası üzerinden bir iş istenebilir. Slayta göre Client bir Visitor olabilir; hoca Visitor ile ilişkiyi daha sonra ayrıca konuşacaklarını söyledi.

11.2 Ebeveyni bilmek ve her yapraktan iletim

İstemci işlemi yapının kök düğümüne iletir ki kökten her alt bileşene iletilebilsin; bu yüzden ilişki Composite'ten Component'e doğrudur. Ama her bileşen ebeveyninin kim olduğunu bilsin isteniyorsa elmas işaretli ilişki çift yönlü düşünülüp yaprağa bir parent üyesi eklenir. Hoca bu üyenin türünün Component değil Composite olması gerektiğini vurguladı: türü Component olsaydı bir çizgiye bir daire ebeveyn olarak atanabilirdi, bu olmaz; Leaf de bir Component olduğu için yaprağa ancak bir Composite ebeveyn olarak verilmelidir.

Ardından yapraktaki bir işlemin yalnız yaprağın kendisinde değil, bütün yapıda yürütülmesi gerekebileceğini tartıştı. Yaprağın Operation'ında önce ebeveynin Operation'ını çağırmak akla gelir, ama bu sonsuz döngüye yol açar. Hoca şemayı birkaç kez çizip sildikten sonra tasarımı ve kodlamayı öğrencilere bıraktı ve şu ipucunu verdi: Composite'te children listesi olduğu gibi her yaprakta da ebeveynine bir bağlantı (myParent) olmalıdır. Kökün ebeveyni null'dır; bir yaprağa bir ileti gelirse önce köke iletilir, kökten bütün düğümlere dağıtılır. Sonsuz döngüye girmemek için her yaprakta "bu komut için işlem yapıldı mı" bilgisini tutan isVisited gibi bir üye alan olmalıdır; bunun başlangıç değerinin nasıl ele alınacağı tasarımcıya kalmıştır.

Hoca bunun bir lüks olduğunu da ekledi: normalde istemci yalnız hiyerarşinin köküyle konuşur denirse bu ek durumları kodlamak gerekmez.

11.3 Uygulanabileceği anlar

- Nesneler arasında hiyerarşik parça-bütün ilişkileri varsa bunları modellemek için.
- Bir nesne topluluğuna ve topluluktaki bireysel üyelere eş biçimli erişim gerektiğinde. Ama bir parçaya gelen isteğin bütün diğer üyelere de iletilmesi isteniyorsa epey ek kod yazmak gerekir.

11.4 Gerçekleme ayrıntıları

- Alt düğüm ebeveyninden haberdar kılınabilir (yukarıdaki myParent üyesi); yukarıdaki senaryo gibi durumlarda yararlı olabilir.
- Bir düğüm nasıl silinecek? Bileşikten bir parça ayrıldığında ya da bir grafik grubu çözülüp parçalara ayrıldığında, ara düğümü ebeveyni mi yoksa hiyerarşinin kökü mü silmeli? Silinecek düğüm bir Composite ise çocukları da silinecek mi? En basiti budur. Silinmemeleri gerekiyorsa başka bir Composite'e aktarılmaları ya da silinenin ebeveynine taşınmaları gerekir.
- Hangi ağaç veri yapısı kullanılacak? Problem hangisine uygunsa o seçilir.

11.5 Zayıf yönler

> **Hocanın vurgusu:** Composite, kalıtımın özüne aykırı davranır. Leaf bir alt sınıftır, ama üst sınıftaki çocuk yönetimiyle ilgili metotlara, kendisinin çocuğu olamayacağı için ihtiyacı yoktur; yine de o metotları boş gövdeyle kodlamak zorundadır.

Ayrıca bu kalıpta programcının karar vermesi gereken gerçekleme ayrıntıları çok fazladır. Çözülen problem bu ayrıntılar hakkında bir şey söylüyorsa gerekenler yapılır; söylemiyorsa programcı yeni kurallar ekleyip kodlamak zorunda kalır. Bu yüzden Composite, tam olarak gerekiyorsa kullanılması gereken bir kalıptır.

12. Örnek Gerçekleme: Makine Üretim Fabrikası

12.1 Problem

Örnek, Design Patterns Java Workbook kitabından değiştirilerek alınmıştır. Bir makine üretim şirketiyiz: çeşitli makineleri ve bu makinelerin parçalarını üretiyoruz, hem yedek parça hem komple makine satıyoruz. Üretimi bilgisayar ortamına aktarıp sorgulamalar ve raporlamalar yapmak istiyoruz: hangi makine hangi fabrikada üretiliyor, alt parçaları hangi fabrikalarda üretiliyor (birden fazla fabrika var). Makine kendi içinde parçalar içerdiği için bir makine hakkında rapor sorgulaması çalıştırıldığında alt parçaları için de yürütülmesi istenir. Arada hiyerarşik bir parça-bütün ilişkisi olduğu için Composite kalıbı kullanılır.

12.2 Tasarım

Slayttaki sınıf şeması (ekrandan, 02:00:00):

```
«abstract» ManufacturingItem
  - machineID : int
  - name, factory : String
  + ManufacturingItem( machineID : int, name : String, factory : String )
  + getMachineID( ) : int
  + getName( ) : String
  + getFactory( ) : String
  + setFactory( String )
  + getParentID( ) : int { abstract }
  + setParentID( int ) { abstract }
  + getParts( ) : Iterator<ManufacturingItem> { abstract }
  + addPart( ManufacturingItem part ) : boolean { abstract }
  + removePart( ManufacturingItem part ) : boolean { abstract }

Machine  (ManufacturingItem'dan türer; ManufacturingItem'larla parça-bütün ilişkisi)
  - parts : ArrayList<ManufacturingItem>
  + Machine( machineID : int, name : String, factory : String )
  + getParentID( ) : int
  + setParentID( int )
  + getParts( ) : Iterator<ManufacturingItem>
  + addPart( ManufacturingItem part ) : boolean
  + removePart( ManufacturingItem part ) : boolean
```

```
MachinePart  (ManufacturingItem'dan türer)
  - parentID : int
  + MachinePart( machineID : int, parentID : int, name : String, factory : String )
  + getParentID( ) : int
  + setParentID( int )
  + getParts( ) : Iterator<ManufacturingItem>
  + addPart( ManufacturingItem part ) : boolean
  + removePart( ManufacturingItem part ) : boolean

Report  --1-----*-->  ManufacturingItem
MainApp  ---->  Report

Not: factory üyesini String yapmak doğru modelleme değil (primitive obsession),
başlı başına sınıf olmalılar. Sadelik için öyle yaptım.
```

Makineler (Machine) ve makine parçaları (MachinePart) soyut ManufacturingItem üst sınıfında toplanır. Her öğenin bir seri numarası (machineID), adı ve üretildiği fabrikanın adı vardır.

> **Hocanın vurgusu:** Fabrikayı yalnız adıyla (String) tutmak bir temel sorundur; her fabrika ayrı bir nesne, dolayısıyla ayrı bir sınıf olmalıydı. Kod kusurları konusuna gelindiğinde görüleceği gibi bu bir Primitive Obsession kusurudur. Hoca örneği odaklı tutmak için böyle yaptığını belirtti.

Kurucu üç alanı ilklendirir. Her üye için getter vardır. Bir parçanın üretildiği fabrika zamanla değişebileceği için factory'nin setter'ı vardır; ad ve ID hiç değişmesin diye onların setter'ı yoktur.

12.3 parentID neden üst sınıfta değil?

Hoca, ManufacturingItem'a bir parentID koyabileceğini, ama örneği basit tutmak için makinelerin alt makinelerden oluşmadığı, yalnız makine ve makine parçasından oluşan iki düzeyli bir hiyerarşi düşündüğünü söyledi. Bir MachinePart'ın hangi ana makinenin içinde olduğunu tutan parentID üyesi vardır; Machine'in parentID'si yoktur, anlamı da yoktur. parentID üst sınıfa konsaydı hem MachinePart'a hem Machine'e kalıtımla aktarılırdı. İki seçenek vardır: ya "anlamı olmayan üye gelsin, kullanmam" denir, ya da kalıtımın özüne ters olan bu davranış Composite'e mümkün olduğunca az dahil edilir. Hoca ikincisini seçti: parentID'yi üst sınıfa eklemedi, bu yüzden getParentID ve setParentID'nin gövdesi doldurulamaz ve soyut bırakılır.

Alt bileşenleri almak için getParts bir Iterator döndürür; her adımda listedeki sıradaki öğeyi gösterir. MachinePart'ın parçaları olmayacak, Machine'in olacaktır; parentID için yapılan tartışma getParts, addPart ve removePart için de geçerlidir, bu yüzden onlar da soyuttur.

12.4 Machine kodu

Eclipse'teki kod (ekrandan, 02:02:20):

```
public Machine( int machineID, String name, String factory ) {
    super(machineID, name, factory);
    parts = new ArrayList<ManufacturingItem>();
}

public int getParentID() { return 0; }
public void setParentID(int parentID) { }

public boolean addPart( ManufacturingItem part ) {
    part.setParentID(getMachineID());
    for( ManufacturingItem aPart : parts )
        if( aPart == part )
            return false;
    return parts.add(part);
}
```

```
public boolean removePart( ManufacturingItem part ) { return parts.remove(part); }
public Iterator<ManufacturingItem> getParts( ) { return parts.iterator(); }
```

Machine'in ebeveyni olmayacağı için getParentID 0 döndürür; böyle bir ID'li nesne aslında yoktur. setParentID hiçbir şey yapmaz. Hoca bu metotları Machine'de soyut tanımlayamayacağını, aksi halde Machine nesnesi oluşturamayacağını; bu yüzden hoş olmasa da böyle kaldığını ve bunun kalıbı eleştirdiği bir nokta olduğunu söyledi.

addPart ise anlamlıdır: makineye bir parça eklenirken önce parçanın ebeveyni bu makine yapılır (makine parçayı sahiplenir). Sonra parça zaten listede var mı diye bakılır; varsa boşuna eklenmez ve false döner, yoksa listeye eklenir. removePart ve getParts, ilgili veri yapısının metotlarını çağırır.

12.5 MachinePart kodu

MachinePart'ın parentID üyesi vardır. Kurucu üst sınıfın kurucusu için gereken bilgileri alıp super'i çağırdıktan sonra parentID'yi atar. parentID için getter ve setter vardır; setter gereklidir, çünkü ana makine parçayı sahiplenirken onu kullanır. Parçanın alt parçası olmayacağı için getParts null döndürür; parça ekleme ve silme metotlarının yapacağı iş yoktur ve false döndürürler. Hoca, her zaman başarılı sayılıp true döndürülmesinin de olabileceğini, yeter ki derlemeyi bozmayacak bir return komutu verilmesi gerektiğini söyledi. Bunlar da kalıbın temel kullanımının bizi kalıtımın özüne ters düşürdüğü noktalardır.

12.6 Rapor

Report sınıfının findItems metodu (ekrandan, 02:05:10):

```
public class Report {
    public static int findItems( ManufacturingItem items[], String factory ) {
        int nr = 0;
        System.out.println("Items manufactured in " + factory + ":");
        System.out.println("Nr.\tID\tName");
        System.out.println("-----------------------------------");
        for( ManufacturingItem item : items ) {
            if( item==null )                    continue;
            if( item.getFactory().compareToIgnoreCase(factory) == 0 ) { //duplicate code -> Re
                nr++;
                System.out.println( nr + ".\t" + item.getMachineID() +
                        "\t" + item.getName() );
            }
            Iterator<ManufacturingItem> subItems = item.getParts();
            if( subItems==null )                    continue;
            while( subItems.hasNext() ) {
                ManufacturingItem part = subItems.next();
                if( part.getFactory().compareToIgnoreCase(factory) == 0 ) { //duplicate code
                    nr++;
                    System.out.println( nr + ".\t" + part.getMachineID() +
                            "\t" + part.getName() );
                }
            }
        }
        // (ekranın devamı görünmüyor)
```

Metot, raporlanacak bütün öğeleri bir dizi olarak ve fabrika adını alır. Başlık, "numara, sekme, ID, sekme, ad" sütun başlıkları ve bir alt çizgi yazar. Dizideki her öğe için: öğe null ise atlanır. Değilse fabrikası alınır; parametre olarak gelen fabrika adıyla büyük-küçük harf farkı gözetmeden eşleşirse bulunan öğe sayısı artırılır ve sıra numarası, ID ve ad yazılır. Bu yetmez, öğenin alt parçaları olabilir; öğenin parçalarına bakılır, parça yoksa geçilir, varsa her parça için aynı fabrika denetimi yapılır.

12.7 Duplicated Code ve Extract Method

Hoca, 13–17. satırlardaki kodla 22–26. satırlardaki kodun neredeyse aynı olduğuna dikkat çekti (altyazıda 19–26 dendi; karede tekrar eden blok 22–26. satırlardadır). Bu, Duplicated Code diye anılan bir kod kusurudur. Çözmek için raporun ReportV2 sürümü yazılmıştır: tekrar eden kısım, ManufacturingItem türünden bir parametre ve fabrika adı alan ayrı bir metoda (showItem) çıkarılmıştır; bu bir Extract Method refactoring işlemidir. Hem Machine hem MachinePart ManufacturingItem'ın alt sınıfları olduğu için tek metot ikisine de yeter. Rapor artık öğe null ise devam eder, değilse öğeyi doğrudan showItem'a iletir; alt parçaları varsa onlar için de showItem çağrılır. Metot yine iki yerden çağrılır, ama kopyala-yapıştır kod kalmamıştır.

Hoca, iki ayrı döngü de olmasın istenirse dönemin sonlarına doğru görülecek Visitor kalıbının bu işi çözdüğünü, ama başka karmaşıklıklar getirdiğini söyledi.

12.8 Çalıştırma

MainApp (ekrandan, 02:10:00, 02:11:10):

```
public static void main(String[] args) {
    ManufacturingItem[ ] items = new ManufacturingItem[4];
    items[0] = new MachinePart(99, 127, "Krank Şaftı KŞ-7", "Bursa");
    Machine jenerator = new Machine(135, "Jeneratör JMX-99", "İstanbul" );
    jenerator.addPart( new MachinePart(259, 135, "Flanş FTR-5", "Bursa") );
    jenerator.addPart( new MachinePart(378, 135, "Kasnak KS-9", "İstanbul") );
    jenerator.addPart( new MachinePart(196, 135, "Rulman RL-3", "Ankara") );
    items[1] = jenerator;
    items[2] = new MachinePart(63, 76, "Distribütör DST-12", "Bursa");
    items[3] = new MachinePart(82, 19, "Pnömatik Vana VPN-7", "Ankara");
    int sayi = Report.findItems(items, "Bursa");
    System.out.println("\t" + sayi + " parça bulundu.");
}
```

Katalog dört elemanlıdır. İlk eleman ID'si 99 olan, 127 ID'li makinenin bileşeni olan Krank Şaftı KŞ-7 adlı bir parçadır ve Bursa'da üretilir. Sonra İstanbul'da üretilen 135 ID'li Jeneratör JMX-99 makinesi tanımlanır ve addPart ile üç parça eklenir: Flanş, Kasnak ve Rulman; bunlar üç ayrı fabrikada üretilir. Parçalar ayrı değişkenlerde tutulmaz, doğrudan addPart'a parametre olarak verilir. Hoca, addPart parçanın ebeveynini zaten atadığı için parçalara 135 vermenin şart olmadığını, okunaklı olsun diye verdiğini söyledi. items[2] ve items[3] yine tekil makine parçalarıdır. Ardından Bursa'da üretilenler listelenir.

Konsol çıktısı:

```
Items manufactured in Bursa:
Nr. ID  Name
-----------------------------------
1.  99  Krank Şaftı KŞ-7
2.  259 Flanş FTR-5
3.  63  Distribütör DST-12
    3 parça bulundu.
```

Krank şaftı bulunmuştur. Jeneratör Bursa'da üretilmese de Bursa'da üretilen bir parçası (Flanş) vardır ve o da bulunmuştur; distribütör de bulunmuştur. ReportV2 de aynı sonucu verir.

Örnek Sorular

Soru 1 (derste sözlü, 00:11:53–00:13:18): Nota programı probleminde Prototype yerine Factory Method ya da Abstract Factory kullanılabilir mi?

Cevap: Hocanın cevabı: Factory Method ile her nota türü için ayrı bir factory metodu gerekir (tek bir fabrika sınıfında da olsa nota türü kadar metot). Abstract Factory ile notalar aile ve üye olarak (örneğin tam notanın yedi, yarım notanın yedi versiyonu) modellenebilir, ama aşırı

sayıda sınıf ortaya çıkar. Bu yüzden Prototype tercih edilir: çerçeve yalnız Graphic'in Clone ve Draw metotlarına dayanır.

Soru 2 (ekrandan, 00:46:40): Bir çizim programında Shape (setLocation, getLocation, display, fill, setColor, undisplay) ve alt sınıfları Point, Line, Square gerçeklenmiştir. Elde metotları setLocation, getLocation, displayIt, fillIt, setItsColor, undisplayIt olan, kaynak kodu olmayan hazır bir XXCircle sınıfı vardır. Daire bu programa nasıl eklenir?

Cevap: Adapter kalıbıyla. Shape'ten türeyen bir Circle sınıfı yazılır ve içine bir XXCircle üyesi konur. Circle'ın display, fill, setColor, undisplay metotları sırasıyla XXCircle'ın displayIt, fillIt, setItsColor, undisplayIt metotlarını çağırır. Circle Adapter, XXCircle Adaptee rolündedir.

Soru 3 (ekrandan, 01:20:00; derste sözlü, 01:22:30–01:24:41): Bridge'siz çözümde dikdörtgen ve daire iki çizim kütüphanesiyle (DP1, DP2) destekleniyor. Üçgen ve ardından üçüncü bir kütüphane (DP3) eklenince kaç somut şekil sınıfı gerekir? Bridge ile kaç sınıf gerekir?

Cevap: Hocanın anlatımına göre Bridge'siz tasarımda her şekil-kütüphane çifti ayrı bir somut sınıftır. İki şekil ve iki kütüphanede $2 \times 2 = 4$ sınıf vardır. Üçgen eklenince Triangle, V1Triangle, V2Triangle gelir ve somut sınıf sayısı $3 \times 2 = 6$ olur. DP3 eklenince V3Rectangle, V3Circle, V3Triangle gelir: $3 \times 3 = 9$. Bridge ile şekil ve çizim hiyerarşileri ayrı olduğu için somut sınıf sayısı toplam olur: üç şekil (Rectangle, Circle, Triangle) ve üç çizim (V1Drawing, V2Drawing, V3Drawing) için $3 + 3 = 6$. Her yeni şekil ya da kütüphane için yalnız bir sınıf eklenir. (Sayılar python ile denetlendi.)

Soru 4 (ekrandan, 02:10:00, 02:11:10): Makine üretim örneğindeki kataloğa `Report.findItems(items, "Bursa")` uygulanırsa çıktı ne olur?

Cevap: Derste çalıştırılarak gösterildi. items[0] Krank Şaftı (Bursa) bulunur. items[1] jeneratörün kendisi İstanbul'da olduğu için bulunmaz, ama parçalarından Flanş FTR-5 (Bursa) bulunur; Kasnak (İstanbul) ve Rulman (Ankara) bulunmaz. items[2] Distribütör (Bursa) bulunur, items[3] Pnömatik Vana (Ankara) bulunmaz. Çıktı: 1. 99 Krank Şaftı KŞ-7, 2. 259 Flanş FTR-5, 3. 63 Distribütör DST-12; "3 parça bulundu." (Sonuç python ile de denetlendi.)

Soru 5 (derste sözlü, 01:48:54–01:50:09): Composite'te istemci herhangi bir yaprağa bir işlem gönderdiğinde işlemin bütün bileşenlerde yürütülmesi isteniyor. Bunu nasıl tasarlarsınız?

Cevap: Hoca tasarımı ve kodlamayı öğrencilere bıraktı, şu ipucunu verdi: Composite'te children listesi olduğu gibi her yaprakta da ebeveyne bir bağlantı (myParent, türü Composite) olmalıdır. Kökün ebeveyni null'dır. Yaprağa gelen ileti önce ebeveynler üzerinden köke iletilir, kökten bütün düğümlere dağıtılır. Aynı düğümün işlemi tekrar yürütüp sonsuz döngüye girmemesi için her düğümde isVisited gibi bir alan tutulur. Hoca, istemcinin yalnız kökle konuşmasının bu ek kodu gereksiz kıldığını da belirtti.

Dersin Toparlaması

Bu derste yaratımsal kalıplar Prototype ve Singleton ile tamamlandı, yapısal kalıplardan Adapter, Bridge ve Composite görüldü. Prototype, ağır sıklet nesneleri sıfırdan kurmak yerine kayıtlı bir prototipi klonlayıp birkaç alanını değiştirmeyi önerir; klonlamada shallow ve deep copy ayrımı ve döngüsel ilişkiler dikkat ister. Singleton, tekil nesnenin sorumluluğunu kendi sınıfına verir (static üye, private kurucu, static getInstance), ama tekil veriyi tek bir sınıf kullanıyorsa basit bir static üye yeterlidir. Adapter, işlevce uyumlu ama imzaca uyumsuz bir hazır sınıfı, hedef arayüzden türeyen ve isteği hazır sınıfa ileten bir ara sınıfla sisteme katar.

Bridge, soyutlama ile gerçeklemeyi ayrı hiyerarşilere koyup aralarını üye alanla bağlayarak, iki boyutta değişen tasarımlarda sınıf sayısının çarpım yerine toplam olarak büyümesini sağlar; "neyin değiştiğini bul ve onu sarmala" ilkesinin akıllıca bir uygulamasıdır. Composite, parça-bütün hiyerarşilerinde tek nesneleri ve bileşikleri istemciye aynı biçimde gösterir, ama yaprakların anlamsız çocuk yönetimi metotlarını taşıması kalıtımın özüne aykırıdır ve birçok gerçekleme kararını programcıya bırakır. Makine fabrikası örneğinde ayrıca Primitive Obsession ve Duplicated Code kusurları ile Extract Method refactoring'i tanıtıldı. Sırada Decorator ile yapısal kalıplara devam edilecektir; Visitor ve Strategy kalıplarına ileride değinileceği de belirtildi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.